

Changelog

All notable changes to **Rev80** are documented here. Sections dated before the 2026-08 rebrand describe the project under its former name, vibechecker, and retain it as an accurate record. Format follows [Keep a Changelog](#).

[Unreleased] — hotfix/claude-ultrareview (2026-08-29)

Integration branch for the three-discipline audit remediation. Sections for the individual branches follow below.

Changed

- **util.py / gui.py** — the **spectral anomaly hook is unwired from the GUI panel**.
`GUI_ANOMALY_HOOK_TYPES` now restricts the monitor dialog's hook combo to `('rms',)`;
`SpectralThresholdHook`, the config schema and the headless front end are untouched, so reviving it is a one-line change
 - It fires on essentially every healthy frame and has never been useful in practice. It triggers on `np.any(|spec - baseline| / baseline > threshold)` across every bin in the band, while Welch runs a single segment in every shipped preset (`nperseg == blocksize`) — so each noise-floor bin is $\chi^2(2)$, with a standard deviation equal to its own mean. $P(\text{some bin of } \sim 2000 \text{ exceeds } 1.5\times)$ is ~ 1.0 on healthy data, which makes `consecutive_n` a delay rather than a defence. Bins 0 and 1 are additionally hard-zeroed for integration, so on any velocity/displacement channel they deviate by $\sim 1e12$ and fire permanently
 - A stored `spectral / both` — which headless still writes — is clamped for display only.
`GUI._hook_type_to_save()` preserves the original, so merely opening the monitor dialog cannot silently rewrite a headless user's configuration; that is the S-07 failure mode this branch had already fixed once
 - Tracked for a fix-or-remove decision as **R39**. A real fix needs band RMS rather than per-bin, a threshold in σ rather than fixed %, and a default band — i.e. it depends on R34
 - **doc/PROGRESS.md** — added **R39** (spectral hook fix-or-remove) and **R40** (sensor-fault detection; records that IEPE bias monitoring is impossible on this hardware — the coupler has a DC blocking capacitor so the bias never reaches the scope, and the 4000A ranges only to ± 20 V against a 24 V supply)
-

[Unreleased] — feature/peak-selection (2026-08-29)

Added

- **src/rev80/peaks.py** — significance-based spectral peak selection, replacing “report the top N local maxima by absolute amplitude”
 - The local noise floor is not flat: across the 60 channel-spectra of the `old castle corpus` it varies by up to $7\times$ *within a single spectrum*. On `blower 4 - bearing DE.h5 ch2` the median local floor is 9.0 mV overall but 39.4 mV over 1890–2000 Hz, so ranking by absolute amplitude ranked by *how loud the neighbourhood is* rather than by *how far a line stands out of it* — 6 of that spectrum's top 12 came from ripple in the noisy top of the band
 - The diagnostic cost was real: 1034 and 1088 Hz are sidebands at $\pm 25/29$ Hz around the 1059 Hz carrier — the bearing-fault signature — and ranked 10th and 4th, so at the shipped display count of

6 the sideband family was broken up and pushed off the table. The instrument was hiding the fault evidence behind noise ripple

- `local_noise_floor()` estimates the floor per bin with a running median, then `select_peaks()` passes **array-valued** height and prominence to a single `find_peaks` call, so admission is evaluated bin by bin against each line's own neighbourhood. Everything that passes is reported; the count becomes an output and the user knob becomes a significance threshold in dB
- **Amplitude reporting is unchanged** — the reported value is still the amplitude of the maximum bin, with no frequency interpolation and no energy summation across a peak. Ranking is still by descending amplitude: significance decides *whether* a line is reported, amplitude decides where it sits in the table
- **tests/test_peak_selection.py** — 626 lines, including an opt-in real-corpus regression suite (`REV80_CORPUS_DIR`, skipped by default since `DEVDATA/` is gitignored)

Changed

- **collector.py** — `process_sample()` step 6 calls `rev80.peaks.select_peaks` instead of `find_peaks(..., distance=5)` + top-N sort. `n_segments` is derived from the Welch parameters rather than assumed to be 1, so the median-to-mean floor correction stays right if `nperseg` ever stops equalling `blocksize`
- **sample.py** — `AcquisitionSettings.peak_threshold_db` (default 9.5), persisted in `to_dict` / `from_dict`
- **gui.py** — the “Peak Display” count spinner is replaced by a “Peak Sig., dB” threshold. The count still exists but only as a clutter cap on the table and plot markers (default 50, against a measured corpus median of 40 and p90 of 49), and a text line reports how many lines passed and whether the cap is hiding any — without it, a capped table is indistinguishable from a spectrum that genuinely had few significant lines

Measured

- **Default threshold 9.5 dB is set by a false-alarm cliff, not taste.** On pure noise (2001 bins, no lines at all) the mean reported count is 37.9 at 6.0 dB, **0.8 at 9.5 dB**, 0.0 at 12.0 dB
- **Floor window width 65 bins** tuned against the corpus, not argued. Scoring single-frame estimators against a 16-frame-averaged reference floor over 20 files × 3 channels × 4 frames gives a broad flat basin from 31 to 65 bins (median |error| 1.097 dB at 31, 1.039 at 45, 1.085 at 65, 1.326 at 129). A *synthetic* spectrum with a deliberately smooth floor prefers 129 — the disagreement is the point
- **Edge handling: truncated window, not the edge replication first proposed.** Replication copies one random bin 32 times, and 32 copies of a single exponential draw dominate a 65-sample median: median |error| 1.480 dB / p95 5.246 dB, against 0.591 / 1.829 dB truncated. Zero-padding (`scipy.signal.medfilt`) is worst, at −3.568 dB mean bias
- Corpus-wide at the default: count median 40, p10 22, p90 49. Under the old top-10 rule 6.67% of reported peaks stood less than 2× above their local floor (18.17% with a flat-top window); the gate admits none
- `wlen` is mandatory with `prominence` — unbounded, a single broad hump spanning the band has prominence 34.6 and its apex looks like the most prominent thing in the spectrum
- Known cost, asserted by a test so it cannot drift silently: **broad features are rejected on purpose.** A 40-bin-σ hump's apex has prominence 7 against a requirement of 144 and never reaches the table. That is right for a line table and wrong if you want broadband resonances flagged; they remain visible in the plot only

[Unreleased] — fix/measurement-validity (2026-08-28)

Nine measurement-validity defects (F-1 ... F-9) from the vibration-engineering audit, plus one adjacent logging defect (S-09).

Root cause of the whole class: the test suite was degenerate. `tests/test_sample.py` sets `F_max=10000`, `binsize=2` → `fs=32768`, `N=16384`, so bin spacing is exactly 2.000 Hz — then swept tones at 500, 1000, 1500 ... 9500 Hz, **every one an exact multiple of 2**. That is the single case where the block is genuinely periodic in `N` samples, the FFT's circular-wrap discontinuity vanishes, and the integration error is identically zero. `_make_dc` also defaulted `highpass_enabled=False`, so the filtered path was never exercised by any amplitude assertion, and the displacement tests asserted only on the spectrum peak, never on `result.overall`. Phase 1 of this branch was therefore a deliberately **red** commit: 110 tests, 92 failing, before any fix was written.

The same mistake was then made again, and caught: every tone in the new suite was generated as `sin(2 $\pi$ ft)`, so **every record started at phase 0** and `x[0]` equalled the DC level exactly — degenerate in precisely the same way bin-centred frequencies are. Real hardware exposed it. `tone()` now takes a `phase` argument defaulting to 0.7 rad.

Added

- `src/rev80/_dsp.py` — windowing helpers for frequency-domain integration, carrying the measured error tables that justify each choice
- `sample.py` — `AcquisitionSettings.nperseg` and `.binsize_actual`: the Welch segment length and the bin width actually delivered
- `collector.py` — `filter_block()`, `filtered_data_for()`, `reset_filter_state()`, `_seed_zi()`: persistent per-channel high-pass state
- `gui.py` — `derive_acquisition_preview()`, a single source of truth for the dialog preview
- `monitor/anomaly.py` — `valid_results()`, a shared validity filter for anomaly hooks
- `tests/test_measurement_validity.py` — 123 tests covering F-1 ... F-9 and S-09, all off-bin, with the high-pass enabled, across block boundaries and at every preset

Changed

- `util.py` — `MAXFREQ_PRESETS` is `[2e2, 5e2, 1e3, 2e3, 5e3, 1e4]`; **the 20 kHz and 50 kHz presets are removed** and `F_max` now tops out at 10 kHz (see F-3)
- `sample.py` — `VibeSample.samplerate` / `ChannelResult.samplerate` / the HDF5 `samplerate` attribute are now `float`. The true rate is generally not an integer
- `sample.py` — `n_fft_bins` counts **displayed** lines (DC...`F_max`), not the full one-sided transform: 1001 at the default preset, not 2049
- `gui.py` — the spectrum info panel reports `binsize_actual`, and is labelled `F_max` rather than `AA`
- `monitor/writer.py` — no longer carries its own divergent copy of `_write_channel_group`
- CRLF → LF in `picoscope.py` and two `examples/` scripts

Fixed

- **F-1** `collector.py` — **FFT wrap leakage corrupted every integrated overall and waveform.** `rfft` was taken on the raw, un-windowed block and multiplied by `(j $\omega$ )n`. The DFT treats the record as periodic; unless it is exactly periodic in `N` samples there is a step discontinuity at the wrap point whose spectrum is broadband and low-frequency-weighted, and `n < 0` amplifies it by `1/ $\omega^{|n|}$` . The in-code claim that the round-trip was lossless held only for `n_ord == 0`, and zeroing bins 0 and 1 left the leakage

in bins 2, 3, 4 ... The spectrum was never affected (Welch already windows); what was wrong was the Overall card, the trend, `overall_json`, the anomaly detector's input, and the displayed waveform

- Two treatments, because the consumers want different things. **Scalar overalls**: Hann taper, RMS divided by the window power gain $\sqrt{\text{mean}(w^2)}$. **Displayed waveform**: cannot be tapered — the envelope would be plainly visible on the trace — so overlap-save instead, with a Tukey $\alpha=0.5$ window killing the wrap discontinuity and only the flat middle, where the window is exactly 1.0, returned. `time_vec` is truncated to match
- **Cost, deliberate and documented**: integrated/differentiated traces span the middle 50% of the block
- Measured (1.0 unit 0-pk sine, $f_s=32768$, $N=16384$) — velocity overall at 61.0 Hz **+12.25%** → **+0.05%**; displacement overall at 61.0 Hz **+470.59%** → **+0.18%**, at 120.7 Hz +777.84% → +0.05%, at 501.0 Hz **+4473.76%** → **+0.00%**; on-bin 500.0 Hz was +0.00% before and after, which is exactly why the old suite never saw any of it. Waveform peak, displacement at 501.0 Hz: +10176.68% → +0.24%
- Taper-width sweep at the worst case (61 Hz displacement): α 0.05 → +335.84%, 0.10 → +7.00%, 0.30 → +4.22%, **0.50** → **+1.47%**. Reflection padding was tried and is far worse (+1918%) — it doubles the effective near-DC content. Differentiation ($n=+1$) verified unaffected, worst +0.018%
- **F-2 `picoscope.py` — the reported sample rate was not the rate actually used.** `_start_streaming` computed `actual_raw_fs` correctly from the interval the driver writes back, then discarded it (`self._actual_samplerate = self.config.samplerate`); the comment had conflated “hide the oversampling ratio” with “hide the actual rate”. The driver quantises the sample interval to whole microseconds, so at $F_{\text{max}}=2000$ the hardware runs 8333.25 Hz while the app labelled it 8192 — **-1.70%**, rising to -8.25% at the (now removed) 50 kHz preset
 - **Confirmed on real hardware** (PicoScope 4424A, AWG loopback on channel A), same captured samples under two labellings: a commanded 1000.00 Hz tone read 983.062 Hz (-1.694%) and now reads 1000.012 Hz (+0.001%). Mean $|\text{error}|$ across four tones **1.676%** → **0.034%**. Reported as a float — rounding would reintroduce a smaller version of the same error
- **F-3 `picoscope.py` — no anti-alias filtering at all above $F_{\text{max}} = 20$ kHz.** `antialias_decimate()` is a no-op at factor 1, and `max(1, int(min(OSR_TARGET, CEILING / samplerate)))` truncated toward zero: 1.526 → 1 at 20 kHz, 0.763 → 0 → clamped to 1 at 50 kHz. The 50 kHz preset additionally requested 131072 Hz raw, **31% above** the measured `STREAMING_CEILING_HZ` — the exact condition the module docstring says makes the driver silently drop most samples while still reporting `status='OKAY'`
 - A general-purpose IEPE accelerometer has a mounted resonance at 25–80 kHz with 20–30 dB of gain, so at $F_{\text{max}}=20$ kHz an unfiltered 50 kHz component folds to 15536 Hz — inside the displayed band, indistinguishable from real signal, and *larger* than the real signal because of the resonance gain
 - `_choose_osr()` now uses explicit `math.floor` and requires `osr >= 2`. Only $F_{\text{max}} \leq 10$ kHz satisfies $f_s * 2 \leq 100$ kHz, hence the preset removal above. Override by raising `STREAMING_CEILING_HZ`, which requires re-measuring the safe continuous streaming rate on the target hardware and channel count. A maxfreq outside the presets still degrades rather than failing, but now logs a WARNING naming the frequency above which content will alias
- **F-4 `collector.py` — high-pass filter state was reset to zero on every block.** `sosfilt` was called with no `zi`, reintroducing a startup transient at the head of every frame of a continuous stream. Measured over blocks 1–3 of a 200 Hz tone at a 10 Hz high-pass: acceleration waveform peak +9.41% → -0.000%, displacement overall +19.15% → +0.017%; with 1000 mV of residual DC offset, acceleration overall **+14321.74%** → **-0.000%**
 - **Two regimes, handled distinctly, because replay is not a stream.** Live streaming filters once per frame in `receive_data`, in order, carrying state, and caches the result on the `VibeSample` since `process_sample` is called repeatedly on the same frame. Replay/browse re-processes stored

- frames out of order and carries no state — verified bit-identical forward and reverse to 12 significant figures. Block 0 of a stream still shows genuine settling; there is no history to carry
- **F-4 follow-up, found by hardware:** `sosfilt_zi(sos) * x[0]` is scipy's documented idiom and is correct when the first sample represents the baseline — true for a step, false for anything oscillatory. On four consecutive real captures the block mean was $-0.06 \dots -0.21$ mV (the true DC) while `x[0]` ranged over **36...305 mV**, and under $1/\omega^2$ that spurious step dominated: worst-block displacement error **+4297.20%** against a settled reference. Now seeded from the block mean (-2.22%). A warm-up pass — filter the block, reuse its final state as its initial state — was measured and is *worse* ($+61.10\%$): it imposes a periodic assumption the block does not satisfy. The branch's own synthetic tests missed this because they used phase-0 sines, reproducing the suite's own central blind spot
 - **F-5 collector.py, monitor/writer.py, monitor/anomaly.py — overload and degraded frames were trended, alarmed on, and stripped on save.** Five gaps forming the classic spurious-alarm mechanism: a clipped waveform reads high with harmonic distortion, the trend records a step change that never happened, the detector fires — and on reload the record looks clean
 1. The overflow bitmask was read from whichever callback completed a block, so a callback raising overflow mid-accumulation had its flag discarded. Now latched with `|=` and cleared on emit
 2. `update_trend` ran regardless of the flags. Flagged frames are now excluded from the trend, while still being displayed and still flagged
 3. `_read_frame_group` hard-coded `overflow=False` and never set `degraded`, so the flags `_write_channel_group` had faithfully stored were never read back
 4. `monitor/writer.py` had a second, divergent `_write_channel_group` that wrote no validity flags at all — the divergence was the defect; they are now one function
 5. The anomaly hooks never read either flag. `valid_results()` is applied to both event evaluation and baseline adaptation: letting a clipped frame into an EWMA baseline poisons the reference for ~ 33 frames just as surely as firing on it
 - **F-6 gui.py — the acquisition dialog computed the sample rate with 2x, not 2.56x.** It duplicated the derivation and used the bare Nyquist minimum, so at the default `F_max=2000` it advertised 4.1 kS/s, 2049 lines, 1.000 s and half the true memory while the instrument ran at 8.2 kS/s, 4097 lines, 0.500 s. Now delegates to `AcquisitionSettings`; verified equal across the full preset grid
 - **F-7 collector.py — the PSD cache key omitted binsize and samplerate.** Switching 2 Hz $\rightarrow$ 0.5 Hz bins returned the identical cached 2049-point, 2 Hz spectrum: the user believed they had quadrupled the resolution and nothing had changed. Masked while streaming, since each new `VibeSample` starts with `psd_mv=None`, so it bit in browse/offline mode and after loading a file
 - **F-8 sample.py — stated line count and bin width did not match the computed spectrum.** `n_fft_bins` returned `blocksize//2 + 1` while Welch used `nfft = int(samplerate/binsize)` and `blocksize = nextpow2(samplerate/binsize) >= nfft`. **40 of 72** preset combinations were wrong; worst `F_max=200/df=20`, a real resolution of 20.480 Hz ($+2.40\%$) and 17 lines claimed against an actual 13. Fixed with `nperseg = blocksize`; re-verified **0 of 72** mismatch. The existing amplitude suite was unaffected — at its `F_max=10000/df=2` the old and new `nperseg` were already both 16384
 - **F-9 gui.py, collector.py — the spectrum was displayed out to fs/2, where alias rejection is ~ 12 dB.** The axis, the peaks table and `find_peaks` all ran to `fs/2 = 1.28 \times F_max` — the anti-alias filter's transition band, -21.8 dB at the folding frequency and effectively 0 dB at `fs/2` itself. The whole point of the `F_max = fs/2.56` convention is that the guard band is never shown. Now truncated at `F_max` before peak-finding. The info label had also called `fs/2` the "AA" frequency, which read as a specification the instrument does not meet
 - **S-09 picoscope.py — the ADC overflow warning inhibit was inverted, flooding the log.** `elif ch in self._overflow_warned: remove(ch)` fired precisely when a channel was *still* clipping, re-arming the warning every other callback — measured **10 warnings from 20 callbacks**. At a 1 ms poll interval that rolls every other diagnostic out of the rotating log during exactly the run being diagnosed. The

clear-down loop also had to move out of `if overflow:`, since a return to zero is the only way to observe it

[Unreleased] — chore/config-consistency-ci (2026-08-28)

Added

- `.github/workflows/ci.yml` — first CI for the project. There was no `.github/` at all: 300+ tests and nothing ran them.
 - `push + pull_request`, `ubuntu-latest`, matrix over Python 3.10–3.13 (the range the new `requires-python` floor admits)
 - `checkout` → `setup-python` → `pip install -e ".[dev]"` → `ruff check src/ tests/` → `pytest tests/ -q`; `fail-fast: false`, concurrency group cancels superseded runs
 - installs `libx11-6`: dearpygui's `_dearpygui.so` links against `libX11` at load time (confirmed with `ldd`). No GL and no X server are needed because the suite never calls `create_viewport()`.
 - the native PicoSDK driver is deliberately **not** installed; the suite runs against `SimulatedSensor`
 - **Verified end to end locally**, not assumed: a clean Python 3.12 venv built exactly as CI does, with the PicoSDK driver stubbed out and `DISPLAY / WAYLAND_DISPLAY` unset, gives 300 passed / 10 skipped. The `picosdk` `git+https` pin resolved and built a wheel with no special handling, so `--no-deps` is not needed. `dearpygui==2.0.0` publishes manylinux wheels for `cp310–cp313` (checked against the PyPI JSON API), so the whole matrix is covered.
- `util.py` — `amplitude_scale(mode)`, `nearest_interval_preset(seconds)`, `canonical_hook_type(value)`, `hook_type_label(value)`, `ANOMALY_HOOK_LABELS`, `DEFAULT_AMPLITUDE_MODE`, `DEFAULT_ANOMALY_HOOK_TYPE`, `DEFAULT_RMS_ALPHA`, `DEFAULT_SPEC_ALPHA`, and an explicit `__all__`
- `util.py` — `600: '10 min'` added to `MONITOR_INTERVAL_PRESETS`
- `README.md` — Contributing section now documents the one-time `git config core.hooksPath .githubhooks` install step
- **Tests** — 7 new files, 135 tests: `test_gui_save_config.py`, `test_icons.py`, `test_monitor_pretrigger_scaling.py`, `test_sensor_library_integrity.py`, `test_config_contract.py`, `test_anomaly_hook_build.py`, `test_util_small_defects.py`

Fixed

- `.githubhooks/pre-commit` — still ran `ruff check vibechecker/` and wrote `vibechecker/_version.py` after the rebrand moved the package to `src/rev80/`. `ruff` exited non-zero against a nonexistent directory, so the hook blocked every commit for anyone who had installed it. Repointed at `src/ tests/` and `src/rev80/_version.py`.
 - `src/rev80/_version.py` regenerated: it read `rc0.4-10-g11a72f4` while `git describe` gives `rc0.5-44-g7bf4712`. **Shipped builds reported a version 34 commits behind, so a field bug report could not be tied to a build.**
 - Root cause of the silent rot: the hook is not installed by default (`core.hooksPath` is the stock `.git/hooks`), now documented in the README.
- `pyproject.toml` — four defects, each breaking a clean install:
 - `requires-python` was `>=3.8`, two minor versions below the real floor. No module uses `from __future__ import annotations`, so every annotation is evaluated at import: `PEP 604 dict | None` in `sensor.py:46` needs 3.10; `dict[str, Any]` in `config.py:219` and

- `argparse.BooleanOptionalAction` in `__main__.py:17` need 3.9. pip installed happily on 3.8/3.9 and the app raised `TypeError` on first import. Raised to `>=3.10`. Swept for 3.11+/3.12+ constructs (`tomllib`, `StrEnum`, `ExceptionGroup`, `except*`, `TaskGroup`, `typing.Self`, `typing.override`, `itertools.batched`, `datetime.UTC`, PEP 695 generics) — none present, so 3.10 is correct, not merely safe.
 - `dearpygui` was an optional `[gui]` extra, but `gui.py:6` and `icons.py:3` import it at module scope and the `rev80` console script reaches both via `__main__:main`. `pip install -e .` — exactly what CLAUDE.md instructs — produced a `rev80` command that `ImportErrors`. Moved into required dependencies; `[gui]` kept as an empty alias.
 - `pandas` and `matplotlib` were declared runtime dependencies but imported nowhere under `src/`. `matplotlib` was already in the `PyInstaller` excludes, confirming it was never needed at runtime. Removed. (Both are still used by the standalone `examples/TMS_Digital_Audio.py` and `scripts/advanced_plots.py`, which sit outside the installed package.)
 - `numpy`, `scipy`, `h5py`, `pyyaml`, `plyer`, `pywin32` were entirely unconstrained, so a shipped installer's contents depended on what PyPI served that day. Added lower bounds (not pins) chosen as the oldest releases that support 3.10 and carry the APIs actually used.
- **pyproject.toml / CI reproducibility** — `[tool.ruff.lint]` set only `ignore`, never `select`, so ruff linted with whatever its *current default* happened to be — and that default moves between releases. On an identical tree: **ruff 0.15.10 → 0 errors, ruff 0.16.5 → 167 errors**, none from a code change. With `ruff` unbounded in `[dev]`, CI would have gone red on an untouched tree at the next ruff release. Fixed at both layers: `select = ["E4", "E7", "E9", "F"]` makes the rule set explicit, and `ruff>=0.15,<0.17` bounds the version. Both versions now report clean.
- **gui.py** — `_on_sb_save_config` called `h5py.File(...)` but `gui.py` never imported `h5py` at module scope; the three other users each did a function-local import and this one did not. The resulting `NameError` was caught by a broad `except Exception` and logged as "failed to patch {session_h5}", so the session browser's **"Save Config" button was dead code** and the message pointed the user at disk permissions. Added the module-scope import, removed the three redundant local ones, and narrowed the `except` to `(OSError, KeyError)`.
- **icons.py / gui.py** — `test_gui_build` failed on any clone that had not run `scripts/build.sh`, with an opaque `SystemError: <built-in function pop_container_stack> returned a result with an exception set`. `assets/fonts/` is gitignored and populated by `build.sh`, so the font is absent on a fresh clone and on every CI runner; `icons.load()` passed the nonexistent path to `dpg.font()`, and the failure inside the context manager surfaced from `pop_container_stack` naming neither the font nor the path — taking down all of `_create_gui()`. Now checks for the file and falls back to the DPG default with a warning. *This was the pre-existing failure standing between the suite and green, and the blocker for CI.*
- **picoscope.py** — the PicoSDK driver (`libps4000a`) is a separate native install from the `picosdk` Python wrapper, and `picosdk/ps4000a.py` instantiates `Ps4000alib()` at import time, raising `CannotFindPicoSDKError` when the driver is absent. `picoscope.py` imported it unguarded, so `import rev80.picoscope` was fatal without the driver. Reproduced by stubbing `find_library`: the **whole suite aborts** with 3 collection errors, not 3 modules' worth of skips — note `test_antialias.py` is pure DSP and was collateral damage. Also contradicted CLAUDE.md's claim of "offline development and CI without hardware". Now degrades to `PICOSDK_AVAILABLE = False`, mirroring the guard `__init__.py:9-17` already uses for this exact import; `FindPicoScope()` returns `[]` with a warning. Verified identical results with the driver present and absent.
- **monitor/controller.py:258** — read the sensor sensitivity under `sensitivity_mv_per_eu`, a key that exists nowhere in the codebase; `session.sensor_snapshot` holds `ScopeSensor.to_dict()` output,

which emits `sensitivity`. The 1.0 default therefore **always** won: the mV → EU division never happened and every burst's pre-trigger trend points came out a factor of `sensitivity` too large (~10x for a 10.2 mV/g sensor) against the post-trigger points on the same continuous plot. `engineering_units` on the adjacent line used the correct key, so the unit *label* converted while the magnitude did not — worse than an obvious break. Now rehydrates through `ScopeSensor.from_dict()` once per call so the field name can only be wrong in one place; misleading comment at :246 corrected.

- **README.md:920 / CLAUDE.md:127** — documented the same wrong `sensitivity_mv_per_eu` key. Following the README raised `KeyError` in `from_dict`, and the very next README paragraph correctly said the code divides by `sensitivity` — the two lines contradicted each other. Both fixed.
- **Sensor library could be silently and permanently destroyed.** `ScopeSensorRegistry._load_user` swallowed any parse failure with `except Exception: return []`, and `_save_user` writes whatever `_load_user` returned straight back over the file. `add()` and `delete()` both follow that load-then-save path, so **one unreadable entry erased every calibrated sensor definition, with nothing logged**. Two independent triggers reached it: the wrong documented key above, and `config._atomic_yaml_write` using `yaml.dump` with the **unsafe default Dumper** while every reader uses `yaml.safe_load` — loading a colleague's `.h5` auto-registers its sensors with no prompt and no type coercion, so a non-scalar attribute serialised as a `!!python/object/apply:` tag that `safe_load` then refused. Fixed at all three layers:
 1. `config.py:159` → `yaml.safe_dump`; unrepresentable values now fail loudly at write time and the existing file survives. Also closes a latent escalation: a writer emitting object tags means any future switch to `yaml.load` becomes arbitrary code execution from a shared measurement file. *No RCE today.*
 2. `scope_sensor.py` → `from_dict` coerces every field and rejects containers and arbitrary objects, so junk cannot reach the writer.
 3. `scope_sensor_registry.py` → a bad *entry* is logged with file, index and content and skipped; a bad *file* raises, so a read failure can never become an overwrite. Read-only callers go through `all()`, which degrades to `[]` and logs, so a corrupt file cannot crash GUI construction.
- **config.py / gui.py / headless.py** — two monitor defaults the GUI could not represent and therefore silently rewrote:
 - `hook_type`: `config.py:83` seeds `'rms'` (lowercase); the GUI combo items are capitalised and both `_on_anom_config_change` and `_build_anomaly_hook` compared raw strings, while `headless.py:159` did `.lower()`. On a fresh install `'rms'` matched neither `('RMS', 'Both')` nor `('Spectral', 'Both')`: opening Config → Monitor once **hid both hook groups, built zero anomaly hooks, and silently disabled anomaly detection** behind an Enable switch that was on. Now canonical lowercase everywhere, capitalisation demoted to a display label.
 - `interval_s`: `config.py:74` seeds `600`, which was not a preset member, so the widget showed “1 h” and saving wrote `3600.0` back — **silently changing a 10-minute logging interval to hourly, discarding 5 of every 6 measurements**. Fixed at both ends: `600` is now a preset, and the fallback picks the *nearest* preset rather than a hardcoded 3600.
- **headless.py / gui.py** — `_build_anomaly_hook` bound `period` only inside the `if hook_type in ("rms", "both")` branch but read it in the spectral branch, so a **Spectral-only config raised UnboundLocalError**. In headless this fires *after* `collector.start_stream()`, killing the process with the PicoScope still streaming and never closed — the worst outcome for an unattended run. The two copies fail differently, which is why it survived: `gui.py` reads `period` unconditionally and always raises, while `headless.py`'s `if "spec_ewma_time" in anom_cfg and period > 0` short-circuits, so it only raises when that key is present — which is exactly what the GUI writes. Hoisted in both.
- **Reconciled the four ways the two `_build_anomaly_hook` copies had drifted:** hook-type casing (above); warmup default 30 (GUI) vs 10 (headless) → 10, matching `config.py`'s seed, since at 30 the

GUI needed a 3x longer baseline warm-up during which nothing could fire; `spec_n` default 3 (GUI) vs 10 (headless) → 10, since at 3 the GUI fired on a third of the evidence headless required; and the EWMA-alpha fallbacks, previously hardcoded separately in each copy, now `util.DEFAULT_RMS_ALPHA / DEFAULT_SPEC_ALPHA`.

- **`_pico_loader.py:24`** — computed `Path(__file__).parent.parent / 'drivers' → src/drivers`, which does not exist; `_paths.py:25` already gets this right with three `.parent` s. `ensure_pico_dlls_loadable()` silently returned `False` in development, bundled DLLs were never registered, and picosdk fell back to walking `%PATH%`. Frozen builds use the `sys._MEIPASS` branch and were unaffected. Now delegates to `_paths.resource_path('drivers')` — the duplication was the bug — and logs a warning on Windows when it returns `False`.
- **`collector.py:1187, 1224`** — deprecated `datetime.utcnow()` replaced with `datetime.now(timezone.utc).replace(tzinfo=None)`, deliberately **naive** to match `utcnow()` 's exact semantics. See follow-ups.
- **`AMPLITUDE_SCALE.get()` fallback inconsistency** — called with two different defaults across five sites: `np.sqrt(2)` in the live path (`collector.py:231, :550`) and `1.0` in the reload path (`collector.py:992, :1147, monitor/controller.py:267`). An unrecognised mode reconstructed a loaded trend **1.414x off** relative to the live trend on the same plot. All five now call `util.amplitude_scale()`; the fallback is `'0-P'` because every call site already normalises with `or '0-P'` before the lookup. Unknown modes are logged. A test asserts no `AMPLITUDE_SCALE.get()` survives anywhere in `src/`.
- **`util.py`** — no `__all__`, so `rev80/__init__.py` 's `from rev80.util import *` re-exported `np` and every imported name into the top-level namespace. Added an explicit `__all__`. `data_dir` is listed deliberately: it is imported into `util` from `rev80._paths` and reached as `rev80.data_dir()` by six call sites in `gui.py / headless.py`, so omitting it would have broken them at runtime rather than at import.
- **18 ruff errors** cleared (unused imports/variables, one `E401`). The five unsafe `F841` s were handled by hand: `test_vibechecker.py:186` 's `old_sr` was kept and *asserted on* — `test_load_offline_adjusts_maxfreq` captured the pre-load samplerate but never compared against it, so the “adjusts” behaviour it is named for went unverified. No `noqa` added anywhere.

Reverted

- **`.python-version`** — briefly changed to `3.13` on the premise that the literal `vibecheck` was a stale string. That premise was wrong: `vibecheck` is a real `pyenv-virtualenv` holding every project dependency, and naming a `virtualenv` there is correct `pyenv-virtualenv` usage. The change resolved to a bare interpreter with no packages and produced 15 collection errors. Reverted; the `pyproject.toml` changes from the same commit stand.

[Unreleased] — refactor/event-pipeline (develop)

Changed

- **`collector.py`** — removed `callbacks` dict entirely; all consumers now read from `frame_cache` via `new_frame_event` rather than receiving samples directly
 - `collect_sample()` rewritten: `new_frame_event.clear() / wait() / frame_cache[-1]` instead of a `_one_shot` closure pinned into `callbacks`
 - `_data_callback()` simplified: appends to cache and sets event; no callback fan-out

- **collector.py** — `data_callback` renamed `_data_callback` (internal-only convention)
- **gui.py** — internal methods renamed with leading underscore: `display_frame` → `_display_frame`, `poll_new_frames` → `_poll_new_frames`, `create_gui` → `_create_gui`
- **gui.py** — removed vestigial `callbacks['plots']` registrations from `_on_load_file`; file-load display now goes through `new_frame_event` → `_poll_new_frames` exclusively, eliminating a latent DPG thread-safety bug (hardware-thread `_display_frame` call)
- **gui.py** — trend plot (`_update_trend_plot`) now called unconditionally in `_display_frame` so loaded HDF5 trend data is rendered in offline browse mode
- **gui.py** — removed two merge-artifact duplicate method definitions: `poll_new_frames` (stale single-quote copy) and `_on_save_click` (old DPG dialog version)
- **gui.py** — removed duplicate `ACQ_NOTES` widget block in `_create_gui` (caused DPG “alias already exists” crash on `test_gui_build`)

Refactored

- **Tests** — `callbacks` references replaced with `frame_cache` reads across `test_vibechecker.py`, `test_multichannel.py`, `test_picoscope.py`, `test_scope_sensor.py`

[Unreleased] — hotfix/hpf-integration-fix (2026-08-28)

Fixed

- **collector.py** — `process_sample()` highpass + integration blow-up (field-reported spurious spike at ~1–2 Hz)
 - Root cause 1: the integration transfer function ($(2\pi f)^n$, applied at all three sites — 5-order overalls, PSD, time-domain output) only zeroed the exact DC bin; bin 1 (1x binsize) received the full multiplier applied to residual near-DC energy and dominated the whole spectrum (0.313 in/s at bin1 vs. a real 0.747 in/s tone peak in a captured reference file)
 - A first fix attempt weighted the transfer function by the highpass filter's frequency response (`scipy.signal.sosfreqz`); this suppressed bin1 by ~50,000x but, being a smooth multiply across many bins rather than an exact single-bin removal, behaved as a wide kernel under circular convolution and badly distorted the time-domain signal at both block edges
 - Fixed instead by hard-zeroing bin1 directly, unconditionally (not gated on `highpass_enabled`) — zeroing an exact FFT bin removes one Fourier basis component losslessly with no boundary sensitivity, the same property that already made the bin0 zero safe
 - Root cause 2 (“edge wobble”, found while investigating #1): the zero-phase highpass (`sosfiltfilt`, introduced in feature/anti-alias) effectively doubles filter order via its forward+backward pass, overshooting the raw signal by 35–45% at both block edges for a low cutoff over a short block (10 Hz over a 1s/4096-sample block); reverted the highpass filter back to causal `sosfilt` (~8–10% startup transient)
 - Verified against real hardware captures (`DEVDATA/hpf-10hz.h5`, 78 Hz / 0.75 in/s-OP test tone): bin1 now exactly 0.0, overall amplitude 0.760 in/s vs. an expected ~0.75, residual edge softness reduced from 350%/246% to ~30–55%
 - **Tests** — 4 new regression tests in `test_sample.py`: bin1 hard-zeroed for integration regardless of `highpass_enabled`, bin1 left untouched for differentiation and passthrough, and a bound on time-domain edge overshoot for the causal highpass
-

[Unreleased] — feature/anti-alias (2026-08-26)

Added

- **picoscope.py** — mandatory anti-alias oversample/decimate stage in `PicoScopeStream`
 - Field incident: a high-frequency bearing-fault harmonic aliased into the low-frequency band at low apparent power, injecting spurious spectral energy; root cause was driving the ADC directly at the target analysis rate with no anti-alias filtering
 - Always oversamples the ADC (`effective_osr` , up to 4x, capped by `STREAMING_CEILING_HZ` — a ceiling measured on real hardware: continuous `ps4000aRunStreaming / GetStreamingLatestValues` silently drops most samples above ~100–250 kHz depending on channel count, with no error indication), applies a zero-phase anti-alias filter, then decimates back to `config.samplerate` via the new `antialias_decimate()` helper — fully transparent to `DataCollector` and everything above it
 - Electrically verified on a PicoScope 4424A with siggen loopback: a tone above target Nyquist that aliased to a spurious 125 mV peak under naive decimation is suppressed 61.9 dB by the real pipeline; an in-band tone near maxfreq passes through unattenuated
- **picoscope.py** — second, independent streaming-rate watchdog flags (`.degraded`) sustained USB throughput below `_RATE_DEGRADED_THRESHOLD` of the requested raw rate — the same silent data-loss mode uncovered during the anti-alias investigation, invisible to the existing silence watchdog since callbacks keep firing with `status='OKAY'` ; deliberately never triggers `_try_recover()` (a USB/bus bandwidth ceiling, not a device hang)
- **sample.py** — `VibeSample / ChannelResult` gain a `degraded: bool` field
- **tests/test_antialias.py** — regression tests for `antialias_decimate()` : aliasing tone suppressed >20x vs. naive decimation, factor=1 no-op, legitimate low-frequency tone survives intact, independent multi-channel handling
- **tests/test_picoscope.py** — `TestRateDegradationWatchdog` : monkeypatched-clock coverage of `_check_rate_degradation()` (healthy/degraded/recovery), confirms `_try_recover()` is never called for this condition

Changed

- **sample.py / util.py** — `samplerate` now derives from `nextpow2(2.56 * maxfreq)` instead of 2x, guaranteeing $\geq 28\%$ Nyquist margin for the anti-alias filter's transition band (matching the ratio commercial FFT vibration analyzers use); `MAXFREQ_PRESETS` drops the 100k/250k/500k Hz entries — hardware measurement showed these already exceed this PicoScope's continuous-streaming ceiling, silently corrupting captured data
- **collector.py** — `receive_data()` threads the `degraded` flag from each incoming frame onto every channel's `VibeSample / ChannelResult` and persists it into saved `.h5` files alongside `overflow` ; new `DataCollector.stream_degraded` property mirrors `is_streaming`
- **collector.py** — `process_sample()` 's lowpass Butterworth block removed (anti-aliasing is now mandatory upstream in `PicoScopeStream`); highpass filter switched from causal `sosfilt` to zero-phase `sosfiltfilt` (reverted in `hotfix/hpf-integration-fix`, below)
- **gui.py** — Lowpass checkbox/field removed from the acquisition dialog; status line shows the auto-derived AA cutoff instead of “LP ...” text; a warning line appears when the stream is degraded

Removed

- User-facing `lowpass_enabled / lowpass_fc` fields — they ran after the ADC had already sampled and so could never actually prevent aliasing; anti-aliasing is now mandatory and handled at capture time

[Unreleased] — rebrand-to-rev80 (2026-08-19 – 2026-08-28)

From this point forward the product is named **Rev80** (formerly vibechecker). Earlier sections below retain the historical “vibechecker” name as an accurate record of the codebase at the time.

Changed

- Rebrand vibechecker → Rev80 across the codebase, packaging, and docs
 - Package moved from `./vibechecker/` to `./src/rev80/` (src-layout; history preserved via `git mv`); all imports updated from `vibechecker` → `rev80`
 - `pyproject.toml`: package name `rev80`, entry points `rev80 / rev80-headless`, `where=["src"]`, `pythonpath=["src"]`
 - User data dir moved to `~/Documents/Rev80/`; config dir moved to `~/.config/rev80/`
 - GUI title/label, CLI prog name, installer (`installer/rev80.iss`), build spec (`rev80.spec`), and icons (`assets/icons/rev80.ico / .svg`) renamed to match; README/PROGRESS docs updated
 - `_paths.py` fix: `resource_path()` 's base needed one more `.parent` after the src-layout move added a directory level (it was resolving into `src/` instead of the project root); `logger.py` now resolves `logging.yaml` directly relative to its own file instead of through `resource_path()`
 - `build/collect_pico_dlls.py` / `build.sh` — search a bundled `vendor/` folder for PicoSDK DLLs first, removing the requirement to install PicoSDK system-wide before building
 - Repo root reorganized: loose docs (`CHANGELOG.md`, `PROGRESS.md`, `VibeGui Project.md`) moved into `doc/`; dev scripts (`build.sh`, `render_progress.sh` → `render_md.sh`, `runtimes.ipynb`) moved into `scripts/`; `rev80.spec` moved into `build/` alongside `collect_pico_dlls.py`, with `ROOT` updated to resolve from `SPECPATH` 's parent so spec-relative paths still point at the repo root; stale `requirements.txt` and `picosdk-install.md` removed
 - `.gitignore` — corrected the PyInstaller build-output path exclusion left stale by the repo reorg
 - Project management docs revisited — outstanding requirements and tech debt items reviewed and updated
-

[Unreleased] — hotfix/welch_leakage (2026-08-18)

Fixed

- `collector.py` — Welch window functions (Hann, Blackman-Harris, etc.) introduce spectral leakage that inflates the reported overall vibration level by a window-dependent factor (Hann: $\sqrt{3/2}$); overall amplitude is now derived from the RMS of an exact-inverse time-domain reconstruction instead of summing spectral peaks
 - The 5-order (`-2 ... +2`) mV RMS overalls are now computed via `irfft` of the integration-scaled `rfft` — reusing the same plain, unwindowed `rfft` already computed once for the time-domain output step, so the round-trip is lossless — as `sqrt(mean(time_ord**2))` instead of `sqrt(sum(psd_ord**2))`
 - The target-unit overall (step 7) now reuses the cached 5-order mV RMS column instead of re-deriving it from the spectrum
 - `collector.py` — `process_sample()` now guards against the resolved integration order falling outside `[-2, 2]`; previously an out-of-range order silently indexed the wrong overalls column, producing false scaling — it now logs an error and returns `None` for the frame
-

[Unreleased] — feature/monitor_mode (2026-05-28 – 2026-06-30)

Added

- **monitor/ package** — Monitor Mode interval datalogger
 - `gate.py`: `IntervalGate` with snap-to-grid scheduling and burst mode
 - `session.py`: `MonitorSession` frozen dataclass (later gains `acq_snapshot / channel_snapshot / sensor_snapshot`, `cooldown_enabled / cooldown_s`)
 - `anomaly.py`: `AnomalyHook` protocol + `NullAnomalyHook` stub
 - `writer.py`: `MonitorWriterThread` (daemon, disk-space guard)
 - `controller.py`: `MonitorController` orchestrating gate/writer/anomaly
 - `gui.py`: Monitor card with config tab and arm/disarm controls (relabelled and reworked repeatedly through the branch — see Changed)
 - 51+ new tests across `test_monitor_gate.py`, `test_monitor_index.py`, `test_monitor_controller.py`
- Session storage iterated through several revisions in-branch:
 - v4: monitor captures write the standard metadata+frames HDF5 layout so `collector.load_data()` can open them directly with no adapter; a `capture_trigger` root attr distinguishes monitor files from manual saves
 - v5 (Phase 2 storage redesign): one `session.h5` per session (`DEVDATA/monitor/{session_id}/`) replaces per-capture files and the SQLite index entirely — `/monitor/{N}/` groups for interval captures, `/burst/{burst_id}/{frame_index}/` groups for burst events, `/burst.attrs['burst_list']` JSON for fast browser rendering without loading frame data
- **Burst capture** — manual (`trigger_burst()`) and anomaly-triggered bursts with a pre-trigger ring buffer
 - Session browser rewritten as a two-tab modal (Monitor / Burst views); `resize_frame_cache()` now expands the cache before load so all captures in a session are browsable, not just the most recent `cache_frames`
 - A run of alignment fixes during the branch: the trigger frame was double-counted in the pre-trigger snapshot; `_burst_all_results` indices were misaligned against pre-trigger frames; burst `rel_time` rebased to the trigger frame (`t=0`) instead of session-relative time; frame cache resized before loading burst frames (previously silently evicted pre-trigger frames at the default 32-frame cache); pre-trigger overalls pre-computed at capture time (`_compute_pretrigger_overalls()`) rather than reprocessed at load; disk-usage estimate corrected to include the pre-buffer, not just burst duration; trend unit conversion restored when browsing a loaded session (sensor wiring was missing on the session-load path, so channels fell back to raw mV)
- **Anomaly detection hooks** (`monitor/anomaly.py`)
 - `RmsThresholdHook` — EWMA self-calibrating per-channel baseline, triggers when `|current - baseline| / baseline` exceeds a threshold for N consecutive frames; tracks streak onset time so `trigger_time / trigger_rel_time` reflect anomaly onset rather than the confirmation frame
 - `SpectralThresholdHook` — initially a stored-baseline bin-by-bin dB comparison with an optional fmin/fmax band; later rewritten to an EWMA per-bin baseline with a percentage threshold and peak-frequency reporting in the trigger reason
 - `CompositeAnomalyHook` — tries each hook in order, returns the first event; propagates `reset_baseline()` to all children
 - `FixedThresholdHook` — independent upper/lower level triggers with unit conversion via `UNIT_TO_SI`
 - `ewma_alpha_from_time(tau, dt)` helper lets EWMA settings be specified as a time constant τ (seconds) instead of the opaque `alpha` value; GUI shows a live computed- α label

- Post-burst cooldown gating replaces arm/disarm entirely: the anomaly hook is supplied once at `start()` and active for the whole session; a cooldown deadline set after any burst fires suppresses further triggers while interval captures continue normally
- GUI: Anomaly Detection section in the Monitor config tab (hook-type combo, RMS/Spectral/Fixed-level/cooldown settings groups, tooltips throughout); “Reset Baseline” replaces the old Arm/Disarm button
- 15+ new tests (`test_monitor_anomaly.py`) covering warmup gating, consecutive-N triggers, baseline set/reset, composite fallthrough, streak tracking, cooldown
- **Headless CLI** (`headless.py`) — interval datalogger without the GUI
 - `python -m vibechecker.headless / vibechecker-headless` console script: discovers a PicoScope (or `--device sim`), loads saved device config, starts the stream, runs `MonitorController`, prints periodic status, and shuts down cleanly on SIGINT/SIGTERM with a session summary
 - `--headless` flag routes `__main__.py` into headless mode; shares the collector/monitor/sample/picoscope pipeline unchanged with the GUI path
 - `--from-file` loads an `.h5` file or monitor session directory on startup (both GUI and CLI); `--[no-]autodetect` controls device discovery, defaulting off when `--from-file` is given
 - `--init-config` seeds `~/.config/vibechecker/` with `acquisition.yaml` and `devices/picoscope-defaults.yaml`
 - Monitor/anomaly settings configured once in the GUI persist to device config and are read back by headless as defaults, overridable by CLI args
 - `dearpygui` made an optional dependency (`pip install -e .` for headless-only installs; `pip install -e '[gui]'` for the GUI)
- **drivers/install-picoscope4000a-driver.sh** — Linux PicoScope driver install script; registers `/opt/picoscope/lib` with `ldconfig` after install
- **gui.py** — Frame info card in the right panel: timestamp, block size, and sample rate always shown; burst-browse-only fields (frame time relative to trigger, burst ID, trigger type/timestamp, max overall per channel); session ID when browsing a session or burst
- **gui.py** — CommitMono Nerd Font icons throughout the left panel (card headers, action buttons, browse-nav arrows); font auto-downloaded on first build and bundled into the frozen app
- **gui.py** — global keyboard shortcuts: Ctrl+A autoscale, Ctrl+K start/stop, Ctrl+S save, Ctrl+O load, Ctrl+Q quit, ←/→ frame browse (live only)
- **gui.py** — configurable frame cache depth (`AcquisitionSettings.cache_frames`, default 32) with derived recording-window and memory-usage display

Changed

- **config.py** — layout split into `acquisition.yaml` (maxfreq/binsize/monitor/anomaly, per instance), `devices/picoscope-<model>-<SN>.yaml` (channels + siggen, per device), and `devices/picoscope-defaults.yaml` (new-device template); `device_config_path()` now takes (`model_name`, `serial_number`) and produces human-readable filenames; no migration path — delete `~/.config/vibechecker` to reseed
- **collector.py** — overalls now always computed over the full FFT spectrum; the `trend_fmin / trend_fmax` “Trend Frequency Window” band-limiting knob removed entirely (dataclass field, config default, UI tags, dialog widgets, tests)
- **collector.py** — `nperseg` clamped to signal length in the Welch PSD call to avoid spurious warnings on short blocks
- **collector.py** — `_load_v3 / _load_v4` consolidated into a single `load_data()` method, fixing a v4 load crash where the old `_load_v4` delegated to `_load_v3`, which read a `data` key that doesn't exist on v4 trend groups

- Version tooling: git-describe version written by a `post-commit` hook, later moved to `pre-commit` so `_version.py` is included in the commit that changes it; installer version now derived from `_version.py` at build time instead of hardcoded in the `.iss` file
- Build fixes accumulated through the branch: UPX disabled (was corrupting the frozen `python3XX.dll`'s PE import table); `pandas` removed (pulled in a `pytz` version-detection failure in frozen builds; the peaks table now uses `list[tuple]`); `plyer`'s Windows filechooser dependency (`win32com / pywintypes`) added to hidden imports; `pyyaml` hidden-import name corrected (`yaml`, not `pyyaml`); `dist/` cleaned and any running instance killed before rebuilding; `assets/` bundled into the frozen app (font was missing, causing a startup crash); PyInstaller cache wipe no longer runs by default (`./build.sh all clean` to force it); Inno Setup arch identifier updated to `x64compatible`

Removed

- `monitor/index.py` (SQLite session index) — superseded by the single- `session.h5` v5 layout
- Arm/disarm as a user-facing concept — anomaly detection now runs for the whole monitored session, gated only by post-burst cooldown

[Unreleased] — refactor/mv-domain-trend (2026-05-05)

Changed

- **sample.py / collector.py** — `VibeSample` now stores raw mV throughout; all sensitivity conversion, Butterworth filtering, and Welch PSD computation moved into `DataCollector.process_sample()`
 - `VibeSample : process(), _convert_time_domain(), push_sample(), save(), load()` removed; adds `overflow: bool` and cached `psd_mv / freq_hz / _psd_config_key / overall_ampl_by_integration_order (5)` mV RMS fields
 - `ChannelResult` gains an `overflow` field
 - `receive_data()` becomes a pure mV pass-through (no sensitivity or filtering applied)
 - `process_sample(ch, sample) : filter → Welch PSD (cached) → 5-order overalls (cached) → sensitivity + SI + integration → ChannelResult`
 - `process_samples()` becomes the collector-owned frame dispatcher; appends to trend only while streaming, uses a `-1` cursor index in browse mode
 - Trend storage changed from a dict-of-lists to `dict[int, {rel_times: ndarray, orders: ndarray(M,5)}]`; new `get_trend_for_display()` centralizes unit/sensitivity/amplitude-mode conversion so `gui.py` no longer imports `util` for it
 - `get_active_eu()` returns `'mV'` immediately when no `ScopeSensor` is assigned, avoiding nonsensical unit conversion before sensitivity is known
 - HDF5 v4: per-channel `(M,5)` orders matrix plus per-channel `rel_times`; v3 files are promoted to v4 trend structure on load (order-0 only)
- **gui.py** — display loop simplified around `process_samples() / get_trend_for_display()`; `_compute_channel_result()` removed; overflow now read from `result.overflow` instead of a bitmask
- **picoscope.py** — `_setup_siggen()` now also called from `start()`; previously it only fired on the reconnect path in `_try_recover()`, so the signal generator never started on the initial `stream.start()` call
- **collector.py** — overalls now computed over the full FFT spectrum unconditionally (groundwork later formalized by removing `trend_fmin / trend_fmax` in `feature/monitor_mode`); HDF5 load paths consolidated

Added

- `gui.py` — global keyboard shortcuts (Ctrl+A/K/S/O/Q, arrow-key frame browse) — later extended in `feature/monitor_mode`
- `gui.py` — configurable frame cache depth with recording-window/memory display, `DEFAULT_CACHE_FRAMES` sourced from `config._BUILTIN_DEFAULTS` — later extended in `feature/monitor_mode`
- Build tooling: git-describe version written by a post-commit hook and embedded into the installer via `_version.py`; PyInstaller/Inno Setup fixes for UPX corruption, a `vibechecker.spec` merge conflict, `pandas / plyer` bundling, and `pyyaml` hidden-import naming

[Unreleased] — feature/windows-build (2026-03-29 – 2026-05-27)

Added

- `_paths.py` — cross-platform path sanitization (Phase 1): `sys._MEIPASS`-aware `data_dir()`, `log_dir()`, `resource_path()`; replaces the third-party `path` library with `stdlib pathlib` across all modules; `SAVEDIR / DataCollector.datadir` route to `~/Documents/vibechecker/data/` in frozen builds
- `_pico_loader.py` / `build/collect_pico_dlls.py` — PicoScope driver bundling (Phase 2): a Windows script locates `ps4000a.dll / picoipp.dll` from the PicoSDK install (registry + default paths), validates the 64-bit PE header, and copies them to `drivers/`; `_pico_loader.py` registers that directory via `os.add_dll_directory()` before `picosdk` import, handling both frozen (`sys._MEIPASS/drivers/`) and dev layouts
- `vibechecker.spec` / `installer/vibechecker.iss` / `build.bat` / `build.sh` — PyInstaller + Inno Setup build pipeline (Phase 3): one-dir PyInstaller build bundling `logging.yaml`, driver DLLs, and dearpygui data; Inno Setup 6 script for a 64-bit, non-admin install to `%LOCALAPPDATA%` with a soft PicoSDK-present check and Start Menu/desktop shortcuts
- App icon: placeholder icon wired into both the PyInstaller spec and Inno Setup script (real artwork deferred)
- `README.md` — restructured for users, contributors, and Windows builders: Windows install section (PicoSDK + installer), cross-platform source install section, Contributing section (dev env, project layout, icon swap guide), full prerequisites table (Python, Git for Windows, PicoSDK, Inno Setup)

Fixed

- PicoSDK 11.x detection by DLL path when the registry key is absent
- `logging.yaml` moved into `vibechecker/` so `resource_path()` resolves it correctly in both dev and frozen builds
- Removed `scipy` submodule excludes from the PyInstaller spec — `scipy.signal` depends on `scipy.linalg` internally and the build broke without it
- Debug `print()` statements stripped from `gui.py / __main__.py`; Inno Setup branding (publisher, copyright, license) updated
- Inno Setup Compiler (`ISCC`) lookup corrected to the proper install `APPDATA` directory
- Icon assets moved to `assets/icons/`

Changed

- **picoscope.py** — `FindPicoScope` replaced with multi-device enumeration via `ps4000aEnumerateUnits`, opening each device by serial number; `PicoScopeStream` gains `_open_unit_by_serial` so it targets the correct device when multiple scopes are connected
- **picoscope.py** — hand-maintained `_channels_for_model` lookup table replaced by `_probe_channel_count`, which calls `ps4000aSetChannel` for channels A–H and counts successes, so any future hardware variant self-reports its channel count without a code change
- Ruff lint fixes across `picoscope.py`, `sample.py`, `util.py`

[Unreleased] — feature/channel-naming (2026-04-03 – 2026-04-13)

Added

- **sample.py** — `AcquisitionSettings` gains `channel_names` and `channel_target_units` dicts with `name_for(ch)` / `target_unit_for(ch)` helpers, persisted in `to_dict` / `from_dict`; later extended with `channel_amplitude_modes`, `channel_couplings`, `channel_voltage_ranges` dicts and matching typed accessors (`amplitude_mode_for`, `coupling_for`, `voltage_range_for`)
- **gui.py** — offline post-analysis mode: `load_data` auto-configures `enabled_channels` and `maxfreq` from file contents; plot series, axes, and results panel sync after load; connection summary shows “File Loaded” (yellow) with frame/channel count; trend plot gets a vertical cursor line for the browsed frame position; acquisition buttons disabled when no device is connected
- **gui.py** — native file dialogs via `plyer.filechooser` (kdialog on KDE, native Win32) replace the DPG file dialog; `SAVEDIR` resolved to an absolute path so the dialog opens in the right place; browse-waveform controls consolidated into a single `_on_browse` dispatcher
- **gui.py** — Channels tab redesign: per-channel collapsing header row (color swatch + name text, then coupling/range/sensor on a second line), themed colored-when-enabled / grey-when-disabled; enabled state moved outside the collapsing header with a target-unit/amplitude indicator on the header itself; amplitude-mode combo added per channel; color indicators switched from a `■` glyph (didn't render in the app font) to `drawlist` / `draw_rectangle`
- **util.py** — expanded `UI_Elements` tag registry: named constants replace previously hardcoded `DEVSETUP_*` / `SREG_FIELD_*` strings; new tags for the redesigned channel rows (`scope_ch_name_text`, `scope_ch_hdr_theme`, `scope_ch_amplitude_mode`)
- File Handling card gains a multiline Measurement Notes widget, read on save and populated on load
- Tooling: `ruff` added to dev dependencies with a `.github/hooks/pre-commit` hook (`git config core.hooksPath .githubhooks` to activate); `pyproject.toml` sets line-length 120, ignores E402/E701

Changed

- **collector.py** — HDF5 format migrated v1 → v2 → v3 across the branch
 - v2: metadata moved into `.attrs` (`/acquisition`, per-frame/per-channel groups) instead of child datasets; `_load_v1` retained for back-compat
 - v3: structured `/metadata/` group — `/metadata/acquisition` (scalars only), `/metadata/scope_sensors/{id}` (sensor library, each unique sensor stored once), `/metadata/channels/{ch}` (name/unit/coupling/voltage_range/sensor_id/target_unit/amplitude_mode); `/frames/{i}/{ch}/data` stores raw samples only; `/trend/re1_times` becomes a shared axis with `/trend/{ch}/data` per channel; `load_data` dispatches to `_load_v3` only, `_load_v1` / `_load_v2` removed; `save_data` overwrites existing files instead of early-returning

- Fixed a file-mode bug where the channel list in the config dialog was derived from `enabled_channels` rather than the frame cache, so disabling a channel made it disappear from the dialog entirely
- **scope_sensor.py** — `amplitude_mode` field removed; it's a per-channel acquisition setting, not a sensor property, and moves to `AcquisitionSettings.channel_amplitude_modes` (old files with the key are silently ignored on load)
- **gui.py** — config dialog scroll fix: outer window gets `no_scrollbar / no_scroll_with_mouse`, the tab bar and each tab's content are wrapped in their own `child_window` so scrolling stays contained per-tab, and the Close button is pinned outside the tab area
- **sample.py** — spectral peak detection: display count default raised (3 → 6) and relabeled "Peak Display"; minimum peak distance changed from a spectrum-length-relative value to a fixed 5 bins

[Unreleased] — refactor/queue-handoff (2026-04-08)

Changed

- **collector.py / gui.py** — decouple collector → GUI with `threading.Event`, precursor to the fuller refactor/event-pipeline cleanup above
 - `DataCollector.new_frame_event`: set by `data_callback` and `reprocess_last_block`
 - `GUI.poll_new_frames`: polls the event each render tick and grabs `frame_cache[-1]`
 - Switched from `dpg.start_dearpygui()` to a manual render loop
 - Removed the `callbacks['plots']` registration — the GUI no longer hooks directly into the collector for display; `collect_sample`'s one-shot capture still used the `callbacks` dict at this point (removed entirely later, in refactor/event-pipeline)
 - Naturally handles GUI lag: rendering always shows the latest frame and skips intermediates, while all frames remain in `frame_cache` for browsing regardless of render rate
- **gui.py** — guarded `poll_new_frames` against a rare `IndexError` race if the hardware thread shifts the deque between `len()` and indexing; silently skips the frame since fresh data arrives next tick
- **README.md / CLAUDE.md** — architecture docs updated for the event-based collector↔GUI decoupling: callback diagrams replaced with the event-signal + poll-loop pattern; stale `sounddevice` references removed

[Unreleased] — feature/picoscope

Added

- **vibechecker/picoscope.py** — PicoScope 4000A acquisition backend (Phase 1)
 - `FindPicoScope()` — enumerates connected PS4000A units; returns `VibeSensor`-compatible dicts with `unit=['mV']`, mirroring the `FindDigiducer` interface
 - `PicoScopeStream` — background polling thread wrapping `ps4000aRunStreaming`
 - Converts ADC counts → mV via `adc2mV` on every driver callback
 - Accumulates variable-sized chunks into exact `blocksize` blocks before firing `DataCollector.receive_data`
 - Implements `.active / .start() / .stop() / .close()` interface (compatible with `sounddevice.InputStream` and `SimulatedSensor`)
 - Handles USB-only / non-USB3 power states (status codes 282 / 286)

- Reads back actual achieved sample rate after `ps4000aRunStreaming` and updates `AcquisitionSettings.samplerate`
- **AcquisitionSettings** — two new PicoScope-specific fields (`sample.py`):
 - `voltage_range: int = 8` — PS4000A range index (8 = PS4000A_5V)
 - `coupling: str = 'AC'` — channel A input coupling ('AC' or 'DC')
- **util.py** — extended `SAMPLERATES` list to include PicoScope-relevant rates: 100 kHz, 200 kHz, 500 kHz, 1 MHz
- **util.py** — added 'mV' to `SUPPORTED_UNITS / UNITS` dict for raw voltage passthrough
- **examples/ps4000a_triangle_stream_plot.py** — standalone script: generates a 500 Hz triangle wave (0.5 V amplitude, +1.4 V DC offset) via the PS4000A signal generator, streams Channel A at 50 kHz for 100 ms, then renders a Plotly HTML report with Welch PSD (10 windows, 50 % overlap) and top-5 peak detection

Changed

- **sensor.py** — `VibeSensor.find()` now calls `FindPicoScope()` instead of `FindDigiducer()`; `VibeSensor.connect()` returns a `PicoScopeStream` for hardware sensors and a `SimulatedSensor` for the simulation path
- **sensor.py** — removed `sounddevice` import and the `sounddevice` reset workaround; renamed internal `_callback` → `_sd_callback` (simulation path only)
- **collector.py** — removed `sounddevice` import; broadened `PortAudioError` catch in `start_stream()` to `Exception`; rewrote `recieve_data()` channel extraction to handle both (N, channels) 2-D arrays and 1-D arrays, with channel index clamping
- **sample.py** — `VibeSample.get_accel()` wraps `convert_units` in a try/except so unsupported conversions (e.g. 'mV' → 'g' before sensitivity is applied) pass through raw data instead of raising
- **README.md** — added PicoScope Integration section: architecture change, new `AcquisitionSettings` fields, Phase 1 data flow diagram, Phase 2 roadmap

Removed

- `digiducer.py` / `sounddevice` no longer used in the main acquisition path (file retained for reference; `FindDigiducer` still exported from `__init__.py`)

[0.1.0] — main (2025-xx-xx)

Initial public snapshot of the **sounddevice** / **Digiducer** acquisition path with:

- `VibeSensor` / `SimulatedSensor` / `DataCollector` pipeline
- `VibeSample` with Welch FFT, velocity spectrum, HDF5 save/load
- `AcquisitionSettings` with enforced interdependencies
- `dearpygui` GUI with real-time time-domain and frequency-domain plots
- Butterworth highpass filter (4th-order SOS, default 10 Hz cutoff)
- Simulated bearing-defect signals (`GenerateBearingVibration_SpectralMethod` , `GenerateBearingVibration_TemporalMethod`)
- Comprehensive README and pytest suite